

n8n Node Catalog & Usage Examples

Table of Contents

- 1. [Trigger Nodes](#)
- 2. [Core Logic Nodes](#)
- 3. [Data Transformation Nodes](#)
- 4. [AI & LangChain Nodes](#)
- 5. [Popular Integration Nodes](#)
- 6. [Database Nodes](#)
- 7. [File & Storage Nodes](#)
- 8. [Communication Nodes](#)

Trigger Nodes

1. Webhook Trigger

Purpose: Receive HTTP requests to start workflows

Common Use Cases:

- API endpoints
- Form submissions
- Third-party integrations
- Real-time event processing

Configuration:

Plain Text
HTTP Method: GET, POST, PUT, DELETE, PATCH Path: /webhook-path Authentication: None, Basic Auth, Header Auth Response: Immediately, Using Respond to Webhook node

Example: Form submission handler

Plain Text

```
Webhook (POST /submit-form)
  → Set (extract form fields)
  → Google Sheets (add row)
  → Send Email (confirmation)
  → Respond to Webhook (success message)
```

2. Schedule Trigger

Purpose: Run workflows on a schedule

Common Use Cases:

- Daily reports
- Data synchronization
- Cleanup tasks
- Periodic checks

Cron Expression Examples:

Plain Text

```
0 9 * * 1-5    → Every weekday at 9 AM
0 */6 * * *    → Every 6 hours
0 0 1 * *      → First day of month at midnight
*/15 * * * *   → Every 15 minutes
```

Example: Daily sales report

Plain Text

```
Schedule (daily at 9 AM)
  → Database (fetch yesterday's sales)
  → Aggregate (calculate totals)
  → Code (format report)
  → Gmail (send to team)
```

3. Manual Trigger

Purpose: Start workflow manually for testing

Use Cases:

- Development and testing

- On-demand tasks
 - Debugging workflows
-

4. Error Trigger

Purpose: Catch errors from other workflows

Configuration:

- Set in main workflow settings: Settings → Error Workflow

Example: Error notification system

Plain Text

```
Error Trigger
→ Set (extract error details)
→ IF (check error type)
  → Critical: Slack (alert team)
  → Non-critical: Database (log error)
```

5. Email Trigger (IMAP)

Purpose: Trigger on incoming emails

Use Cases:

- Email processing
- Support ticket automation
- Invoice extraction

Example: Support ticket creation

Plain Text

```
Email Trigger (IMAP)
→ IF (check subject contains "Support")
→ Extract Data (parse email content)
→ HTTP Request (create ticket in system)
→ Gmail (send confirmation)
```

Core Logic Nodes

1. IF Node

Purpose: Branch workflow based on conditions

Condition Types:

- String: equals, contains, starts with, ends with
- Number: equals, greater than, less than
- Boolean: true/false
- Date: before, after
- Exists: field exists or not

Example: Customer segmentation

Plain Text

```
IF ({{ $json.orderTotal > 1000 }})
  → True: Set (VIP customer)
  → False: Set (Regular customer)
```

2. Switch Node

Purpose: Multiple conditional branches

Use Cases:

- Complex routing
- Status-based processing
- Multi-tier logic

Example: Order status routing

Plain Text

```
Switch ({{ $json.status }})
  → Case "pending": Send reminder
  → Case "paid": Process order
  → Case "cancelled": Refund
  → Default: Log unknown status
```

3. Code Node

Purpose: Execute JavaScript or Python code

Available Variables:

- `$input` : Input data
- `$json` : Current item JSON
- `items` : All items
- `$node` : Access other nodes
- `$workflow` : Workflow info

JavaScript Example: Data transformation

JavaScript

```
// Transform array of objects
const items = $input.all();

return items.map(item => ({
  json: {
    fullName: `${item.json.firstName} ${item.json.lastName}`,
    email: item.json.email.toLowerCase(),
    createdAt: new Date().toISOString()
  }
}));
```

Python Example: Data analysis

Python

```
import statistics

# Calculate average from items
values = [item['json']['value'] for item in items]
average = statistics.mean(values)

return [{'json': {'average': average, 'count': len(values)}}]
```

4. Merge Node

Purpose: Combine data from multiple branches

Merge Modes:

- **Append:** Combine all items
- **Keep Key Matches:** Match items by key field
- **Multiplex:** Create combinations

- **Wait:** Wait for all inputs

Example: Enrich customer data

Plain Text

```
Branch 1: Database (get customer info)
Branch 2: API (get purchase history)
  → Merge (Keep Key Matches on customerID)
  → Set (combine data)
```

5. Split In Batches Node

Purpose: Process large datasets in chunks

Configuration:

- Batch Size: Number of items per batch
- Options: Reset, Batch Index

Example: Bulk email sending

Plain Text

```
Database (get 1000 subscribers)
  → Split In Batches (100 items)
  → Loop Over Items
    → Gmail (send email)
    → Wait (1 second delay)
  → Loop back to Split In Batches
```

6. Loop Over Items Node

Purpose: Process each item individually

Use Cases:

- Item-by-item processing
- API calls for each item
- Sequential operations

Data Transformation Nodes

1. Set Node

Purpose: Create or modify fields

Operations:

- Add fields
- Set values from expressions
- Copy data from other nodes

Example: Prepare API payload

Plain Text

Set:

- name: {{ \$json.firstName }} {{ \$json.lastName }}
- email: {{ \$json.email.toLowerCase() }}
- timestamp: {{ \$now.toISO() }}
- source: "n8n automation"

2. Edit Fields Node

Purpose: Rename, remove, or keep specific fields

Operations:

- Remove fields
- Rename fields
- Keep only specific fields

Example: Clean API response

Plain Text

Edit Fields:

Remove: `_id`, `__v`, `createdAt`, `updatedAt`
Rename: `user_name` → `name`, `user_email` → `email`

3. Aggregate Node

Purpose: Summarize and group data

Aggregation Types:

- Count, Sum, Average, Min, Max

- Group by field
- Concatenate strings
- Collect into array

Example: Sales summary

Plain Text

Aggregate:

- Aggregate: Sum
- Field to Aggregate: orderTotal
- Group by: salesPerson

4. Filter Node

Purpose: Remove items that don't match conditions

Example: Filter active customers

Plain Text

Filter:

Condition: `{{ $json.status === 'active' && $json.lastOrderDate > $now.minus({days: 90}) }}`

5. Sort Node

Purpose: Sort items by field

Example: Sort by date descending

Plain Text

Sort:

Field: createdAt
Order: Descending

AI & LangChain Nodes

1. AI Agent

Purpose: Create AI agents with tools and memory

Components:

- Chat Model (OpenAI, Anthropic, etc.)
- Tools (sub-nodes)
- Memory (conversation history)
- Prompt template

Example: Customer support agent

Plain Text

AI Agent:

- Chat Model: OpenAI GPT-4
- Tools:
 - Search Knowledge Base
 - Create Ticket
 - Check Order Status
- Memory: Buffer Memory

2. OpenAI Chat Model

Purpose: Generate AI responses

Models:

- GPT-4, GPT-4 Turbo
- GPT-3.5 Turbo

Example: Content generation

Plain Text

OpenAI Chat Model:

Prompt: Generate a product description for {{ \$json.productName }}

Temperature: 0.7

Max Tokens: 500

3. Vector Store (Pinecone, Supabase, etc.)

Purpose: Store and search embeddings

Use Cases:

- Semantic search

- RAG (Retrieval Augmented Generation)
 - Document similarity
-

4. Document Loaders

Purpose: Load documents for AI processing

Types:

- PDF, Text, CSV
 - Web pages
 - GitHub files
-

Popular Integration Nodes

1. HTTP Request

Purpose: Make API calls to any service

Methods: GET, POST, PUT, DELETE, PATCH

Authentication:

- None
- Basic Auth
- Header Auth
- OAuth2
- Custom

Example: REST API call

Plain Text

```
HTTP Request:
Method: POST
URL: https://api.example.com/users
Headers:
  - Authorization: Bearer {{ $credentials.apiKey }}
Body:
{
  "name": "{{ $json.name }}",
```

```
"email": "{{ $json.email }}"  
}
```

2. Google Sheets

Purpose: Read/write Google Sheets

Operations:

- Append row
- Update row
- Get rows
- Delete row
- Clear sheet

Example: Log data to sheet

Plain Text

```
Google Sheets:  
  Operation: Append  
  Spreadsheet: Sales Tracker  
  Sheet: 2025  
  Columns: Date, Customer, Amount, Status
```

3. Airtable

Purpose: Interact with Airtable bases

Operations:

- Create, Read, Update, Delete records
- List records with filters

4. Notion

Purpose: Manage Notion pages and databases

Operations:

- Create/update pages
- Query databases
- Append blocks

Database Nodes

1. PostgreSQL / MySQL

Purpose: Execute SQL queries

Operations:

- Execute Query
- Insert, Update, Delete

Example: Fetch users

Plain Text

PostgreSQL:

Operation: Execute Query

Query: `SELECT * FROM users WHERE status = 'active' AND created_at > NOW()
- INTERVAL '30 days'`

2. MongoDB

Purpose: NoSQL database operations

Operations:

- Find, Insert, Update, Delete
- Aggregate

3. Supabase

Purpose: Interact with Supabase (PostgreSQL + APIs)

Features:

- Database operations
- Authentication
- Storage
- Real-time subscriptions

File & Storage Nodes

1. Read/Write Files from Disk

Purpose: Local file operations

Operations:

- Read file
 - Write file
 - Move file
-

2. Google Drive

Purpose: Manage Google Drive files

Operations:

- Upload, Download
 - Create folder
 - Share file
 - Search files
-

3. AWS S3

Purpose: Object storage operations

Operations:

- Upload, Download
 - List, Delete
 - Generate presigned URLs
-

4. Dropbox

Purpose: Dropbox file management

Communication Nodes

1. Gmail

Purpose: Send and manage emails

Operations:

- Send email
- Get emails
- Mark as read
- Add label

Example: Send formatted email

Plain Text

```
Gmail:
  Operation: Send
  To: {{ $json.email }}
  Subject: Order Confirmation #{{ $json.orderNumber }}
  Body (HTML):
    <h1>Thank you for your order!</h1>
    <p>Order total: ${{ $json.total }}</p>
```

2. Slack

Purpose: Send messages and interact with Slack

Operations:

- Send message
- Update message
- Get channel history
- Upload file

Example: Alert notification

Plain Text

```
Slack:
  Operation: Post Message
  Channel: #alerts
  Text: ⚠ Error in workflow: {{ $json.error }}
  Blocks: [Formatted message with buttons]
```

3. Telegram

Purpose: Telegram bot integration

Operations:

- Send message
 - Send photo/document
 - Edit message
-

4. Discord

Purpose: Discord bot operations

5. Twilio

Purpose: SMS and voice calls

Operations:

- Send SMS
 - Make call
 - Send WhatsApp message
-

Advanced Patterns

Pattern 1: Retry with Exponential Backoff

Plain Text

HTTP Request:

Settings:

- Retry On Fail: Yes
- Max Tries: 5
- Wait Between Tries: 1000ms
- Wait Time Multiplier: 2

Pattern 2: Parallel Processing

Plain Text

Start

- Branch 1: Process Type A
- Branch 2: Process Type B
- Branch 3: Process Type C

- Merge (Wait for all)
- Continue

Pattern 3: Error Recovery

Plain Text

Main Operation (Continue On Fail: OFF)

- Success Path

Error Workflow:

- Error Trigger
- IF (error.type === 'timeout')
 - Retry Operation
- ELSE
 - Alert Admin

Pattern 4: Rate Limiting

Plain Text

Loop Over Items

- API Call
- Wait (1 second)
- Continue

Best Practices by Node Type

HTTP Request

-  Always handle errors (Continue On Fail or Error Workflow)
-  Use retry logic for external APIs
-  Store credentials securely
-  Add timeout settings
-  Don't hardcode API keys

Code Node

-  Use for complex transformations
-  Add comments for clarity

-  Handle errors with try-catch
-  Return proper item format
-  Don't use for simple operations (use Set instead)

Database Nodes

-  Use parameterized queries
-  Limit result sets
-  Index frequently queried fields
-  Use transactions for multiple operations
-  Don't fetch all data if you only need some

AI Nodes

-  Set appropriate temperature
 -  Limit token usage
 -  Cache results when possible
 -  Add fallback logic
 -  Don't send sensitive data without encryption
-

Quick Node Selection Guide

Need to...

- **Start workflow on schedule** → Schedule Trigger
- **Receive HTTP requests** → Webhook Trigger
- **Make API calls** → HTTP Request
- **Transform data** → Code, Set, Edit Fields
- **Branch logic** → IF, Switch
- **Combine data** → Merge
- **Process large datasets** → Split In Batches
- **Send emails** → Gmail, Send Email
- **Store data** → Database nodes, Google Sheets
- **Use AI** → AI Agent, OpenAI Chat Model
- **Handle errors** → Error Trigger, Stop And Error

This catalog covers the most commonly used nodes. For a complete list and detailed documentation, visit: <https://docs.n8n.io/integrations/builtin/>